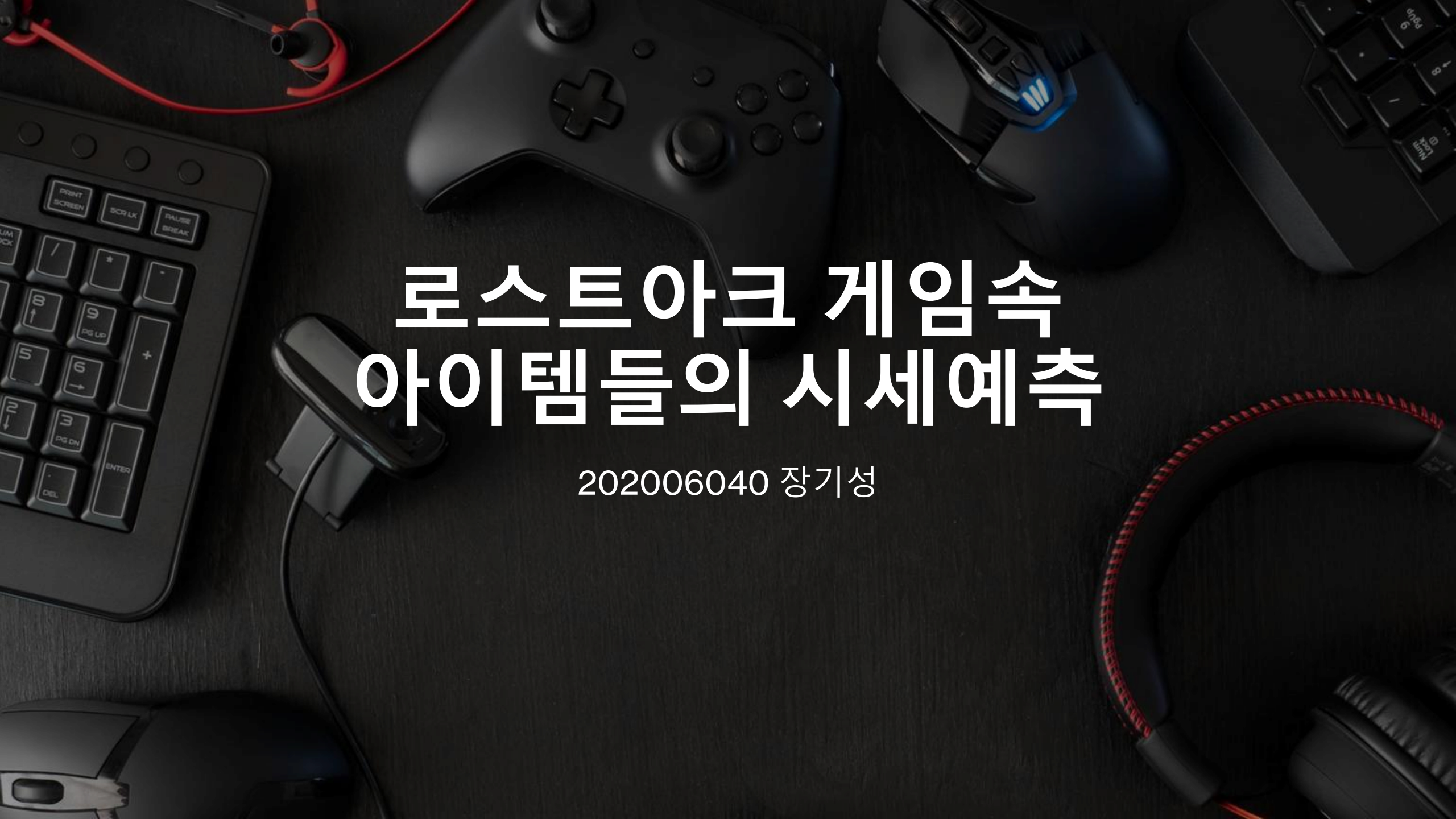




로스트아크 게임속 아이템들의 시세예측

202006040 장기성

목차

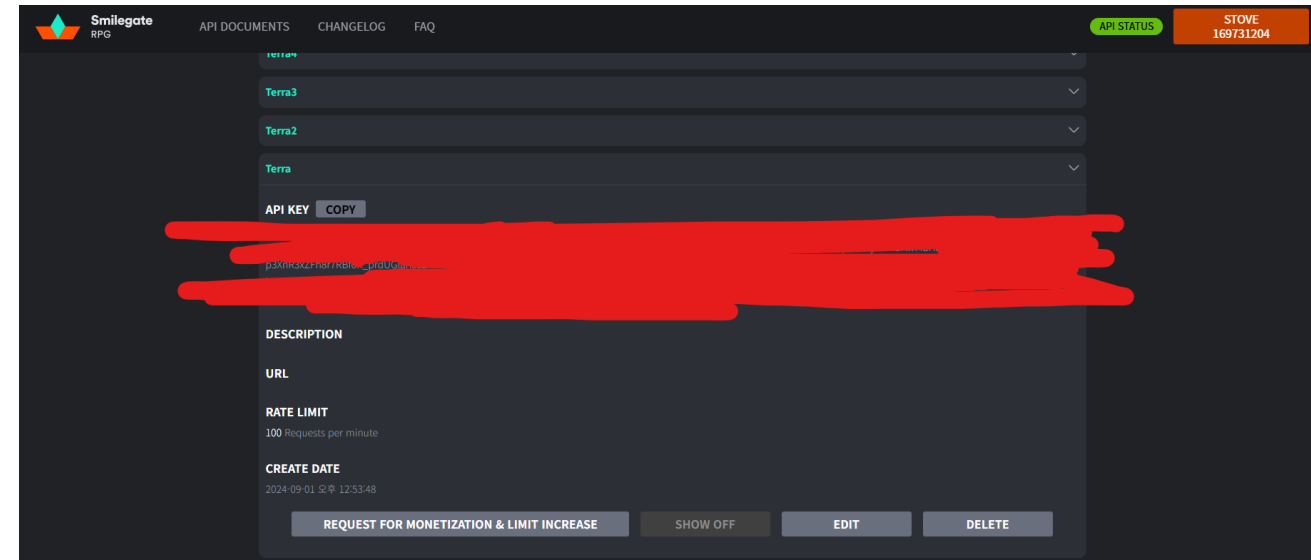
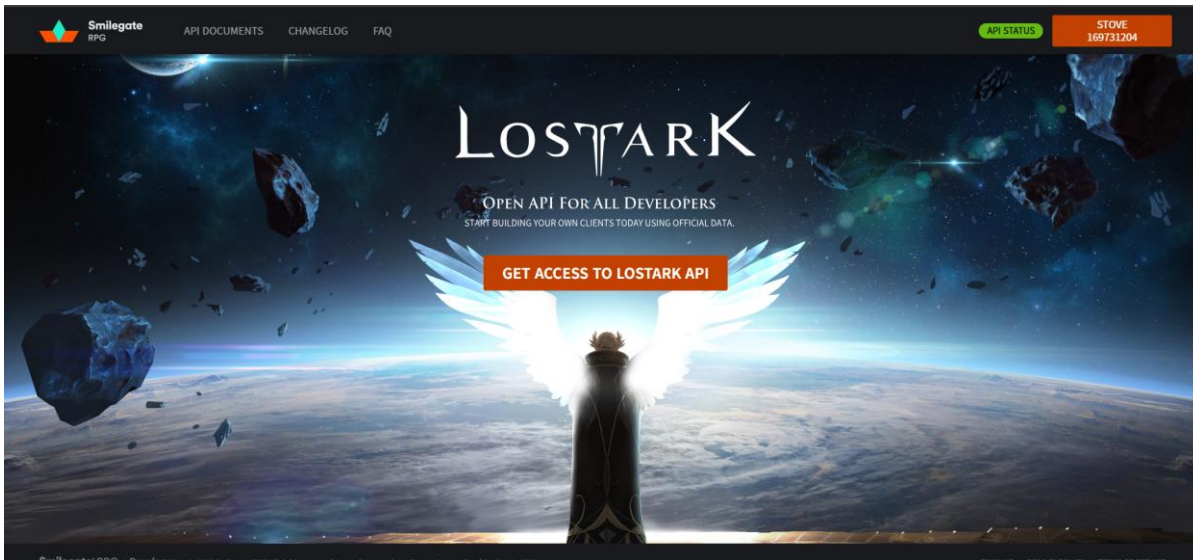
- 프로젝트 동기
- 데이터 수집 과정
 - 데이터 수집 시스템 구축과정
- 전처리
- 학습
- 결과(시각화 포함)
- 기대효과

프로젝트 동기

- 게임내 재화가 많이쌓여서 이를 소모해 캐릭터를 강화하려고
- 강화에 필요한 아이템을 구매하였는데
- 다음날 보니 해당 아이템의 가격이 훨씬 저렴했던것
- -> 아이템의 미래의 가격흐름양상을 예측해주면 좋을것 같다는 생각이 듦

데이터 수집 과정

- 데이터를 요청하기 위한 API키 발급



데이터 수집 과정

- 공식 문서를 참고

The screenshot shows the Smilegate RPG API documentation interface. The top navigation bar includes the Smilegate RPG logo, links to API DOCUMENTS, CHANGELOG, and FAQ, and status indicators for API STATUS and STONE 169731204. The left sidebar shows the 'MARKETS' section selected. The main content area displays the GET endpoint `/markets/items/{itemid}` with a description 'Returns a market item by ID.' Below this, the POST endpoint `/markets/items` is also listed. The 'Parameters' section is expanded, showing a table with columns 'Name' and 'Description'. The 'requestMarketItems' parameter is detailed with its type 'object' and an example JSON value. The 'Responses' section is partially visible at the bottom.

Name	Description
requestMarketItems	The search options for the market

```
{  "Sort": "GRADE",  "CategoryCode": 0,  "CharacterClass": "string",  "ItemTier": null,  "ItemGrade": "string",  "ItemName": "string",  "PageNo": 0,  "SortCondition": "ASC"}
```

The screenshot shows the Smilegate RPG API documentation interface, displaying the JSON structure for the `CategoryItem` and `MarketItem` endpoints. The top navigation bar and sidebar are consistent with the previous screenshot. The main content area shows the JSON structure for the `CategoryItem` endpoint, which includes a `Subs` array and a `CategoryItem` object. The `MarketItem` endpoint structure is also shown below.

```
{  "Subs": [    {      "Code": "integer(Sint32)",      "CodeName": "string"    }  ],  "CategoryItem": {    "Id": "integer(Sint64)",    "Name": "string",    "Grade": "string",    "Icon": "string"  }  }
```

데이터 수집 과정

- 테스트용 프로그램 제작



Program.cs

```
using System.Net.Http;
using System.Net.Http.Headers;
using System.Text;
using System.Threading.Tasks;
using Newtonsoft.Json.Linq;

namespace LottakgiTest
{
    internal class Program
    {
        static string key = "B4 Pw dW4GdF1";
        static string[] para = { "", "20000", "", "", "00", "", "", "" };

        static async Task Main(string[] args)
        {
            string url = "";
            string name = { "Sort", "CategoryCode", "CharacterClass", "ItemLine", "ItemGrade", "ItemName", "People", "SortCondition" };

            while (true)
            {
                Console.WriteLine("Input url : ");
                url = Console.ReadLine();
                if (url == "") { break; }
                if (url == "q")
                {
                    url = "http://developer-lottakgi.com.swagger.io/swagger/index.html";
                    while (true)
                    {
                        Console.WriteLine("Input name : ");
                        string name = Console.ReadLine();
                        if (name == "q")
                        {
                            Print("q.Sort : {para[0]}");
                            Print("q.CategoryCode : {para[1]}");
                            Print("q.CharacterClass : {para[2]}");
                            Print("q.ItemLine : {para[3]}");
                            Print("q.ItemGrade : {para[4]}");
                            Print("q.ItemName : {para[5]}");
                            Print("q.People : {para[6]}");
                            Print("q.SortCondition : {para[7]}");
                        }
                        else if (int.TryParse(name, out int index))
                        {
                            if (index < 0 || 7 < index) { continue; }

                            Console.WriteLine("{name[index]} : ");
                            para[index] = Console.ReadLine();
                        }
                        else if (name == "q")
                        {
                            url = "http://developer-lottakgi.com.swagger.io/swagger/index.html";
                            break;
                        }
                        else if (name == "qr")
                        {
                            url = "http://developer-lottakgi.com.swagger.io/swagger/index.html";
                            break;
                        }
                    }
                }
            }

            HttpClient client = new HttpClient();
            {
                client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("bearer", key);
                client.DefaultRequestHeaders.Accept = new MediaTypeWithQualityHeaderValue("application/json");
                string jsonData = "";
                try
                {
                    jsonData = JsonConvert.SerializeObject(new ItemSearchRequest()
                    {
                        Sort = para[0],
                        CategoryCode = int.Parse(para[1]),
                        CharacterClass = para[2],
                        ItemLine = para[3],
                        ItemGrade = para[4],
                        ItemName = para[5],
                        People = para[6],
                        SortCondition = para[7]
                    });
                }
                catch
                {
                    Print("Error msg");
                    continue;
                }
                Print(jsonData);
                HttpResponseMessage response = await client.GetAsync(url, HttpContent);
                string body = await response.Content.ReadAsStringAsync();
                JObject root = JObject.Parse(body);
                body = JsonConvert.SerializeObject(JsonConvert.DeserializeObject(body), Formatting.Indented);
                Print(body);
            }
        }

        static void Print(object obj)
        {
            Console.WriteLine(obj);
        }

        public class ItemSearchRequest
        {
            public string Sort { get; set; }
            public int CategoryCode { get; set; }
            public string CharacterClass { get; set; }
            public string ItemLine { get; set; }
            public string ItemGrade { get; set; }
            public string ItemName { get; set; }
            public string People { get; set; }
            public string SortCondition { get; set; }
        }
    }
}
```

데이터 수집 과정

- API 테스트

```
E\WlostarkApiTestWlostarkAp x + v - □ x
input url : p
input menu : h
0.Sort :
1.CategoryCode : 230000
2.CharacterClass :
3.ItemTier :
4.ItemGrade : 영웅
5.ItemName :
6.PageNo : 1
7.SortCondition :
input menu : 0
Sort : BUY_PRICE
input menu : 1
CategoryCode : 210000
input menu : 3
ItemTier : 4
input menu : 5
ItemName : 10레벨
input menu : 7
SortCondition : ASC
input menu : h
0.Sort : BUY_PRICE
1.CategoryCode : 210000
2.CharacterClass :
3.ItemTier : 4
4.ItemGrade : 영웅
5.ItemName : 10레벨
6.PageNo : 1
7.SortCondition : ASC
input menu : 4
ItemGrade :
input menu : h
0.Sort : BUY_PRICE
1.CategoryCode : 210000
2.CharacterClass :
3.ItemTier : 4
4.ItemGrade :
5.ItemName : 10레벨
6.PageNo : 1
7.SortCondition : ASC
input menu : q
```

```
E\WlostarkApiTestWlostarkAp x + v - □ x
input url : p
input menu : 00
{"Sort": "BUY_PRICE", "CategoryCode": 210000, "CharacterClass": "", "ItemTier": "4", "ItemGrade": "", "ItemName": "10레벨", "PageNo": 1, "SortCondition": "ASC"}
{
  "PageNo": 1,
  "PageSize": 10,
  "TotalCount": 42,
  "Items": [
    {
      "Name": "10레벨 작렬의 보석",
      "Grade": "고대",
      "Tier": 4,
      "Level": 1640,
      "Icon": "https://cdn-lostark.game.onstove.com/efui_iconatlas/use/use_12_115.png",
      "GradeQuality": null,
      "AuctionInfo": {
        "StartPrice": 2330000,
        "BuyPrice": 2330000,
        "BidPrice": 0,
        "EndDate": "2025-11-12T21:01:57.27",
        "BidCount": 0,
        "BidStartPrice": 2330000,
        "IsCompetitive": false,
        "TradeAllowCount": 0,
        "UpgradeLevel": null
      },
      "Options": [
        {
          "Type": "GEM_SKILL_COOLDOWN_REDUCTION",
          "OptionName": "정글진",
          "OptionNameIripod": "",
          "Value": 24.0,
          "IsPenalty": false,
          "ClassName": "정글사",
          "IsValuePercentage": true
        }
      ]
    },
    {
      "Name": "10레벨 작렬의 보석",
      "Grade": "고대",
      "Tier": 4,
      "Level": 1640,
      "Icon": "https://cdn-lostark.game.onstove.com/efui_iconatlas/use/use_12_115.png",
      "GradeQuality": null,
      "AuctionInfo": {
        "StartPrice": 2330000,
        "BuyPrice": 2330000,
        "BidPrice": 0,
        "EndDate": "2025-11-12T21:02:02.347",
        "BidCount": 0,
        "BidStartPrice": 2330000,
        "IsCompetitive": false,
        "TradeAllowCount": 0,
        "UpgradeLevel": null
      },
      "Options": [
        {
          "Type": "GEM_SKILL_COOLDOWN_REDUCTION",
          "OptionName": "정글포",
          "OptionNameIripod": "",
          "Value": 24.0,
          "IsPenalty": false,
          "ClassName": "정글사",
          "IsValuePercentage": true
        }
      ]
    },
    {
      "Name": "10레벨 작렬의 보석",
      "Grade": "고대",
      "Tier": 4,
      "Level": 1640,
      "Icon": "https://cdn-lostark.game.onstove.com/efui_iconatlas/use/use_12_115.png",
      "GradeQuality": null,
      "AuctionInfo": {

```

데이터 수집 과정

- 테스트 결과
- 수집 가능한 데이터
 - 1. 아이템의 현재 최저가
 - 2. 특정 아이템의 최대 2주전까지의 하루합산 데이터
 - 평균판매가, 최저판매가

데이터 수집 과정

목표로 하는 모델을 학습시키려면
분단위 데이터가 필요

하지만 과거 데이터는 하루평균
데이터뿐

-> 데이터 수집 시스템을 구축
하기로 결정

데이터 수집 시스템 구축과정

- 지인의 도움을 받아서 거래량이 높은 아이템을 선별함
- 강화재료, 강화보조재료, 보석, 켄(보석이랑 다름), 각인서, 생활재료

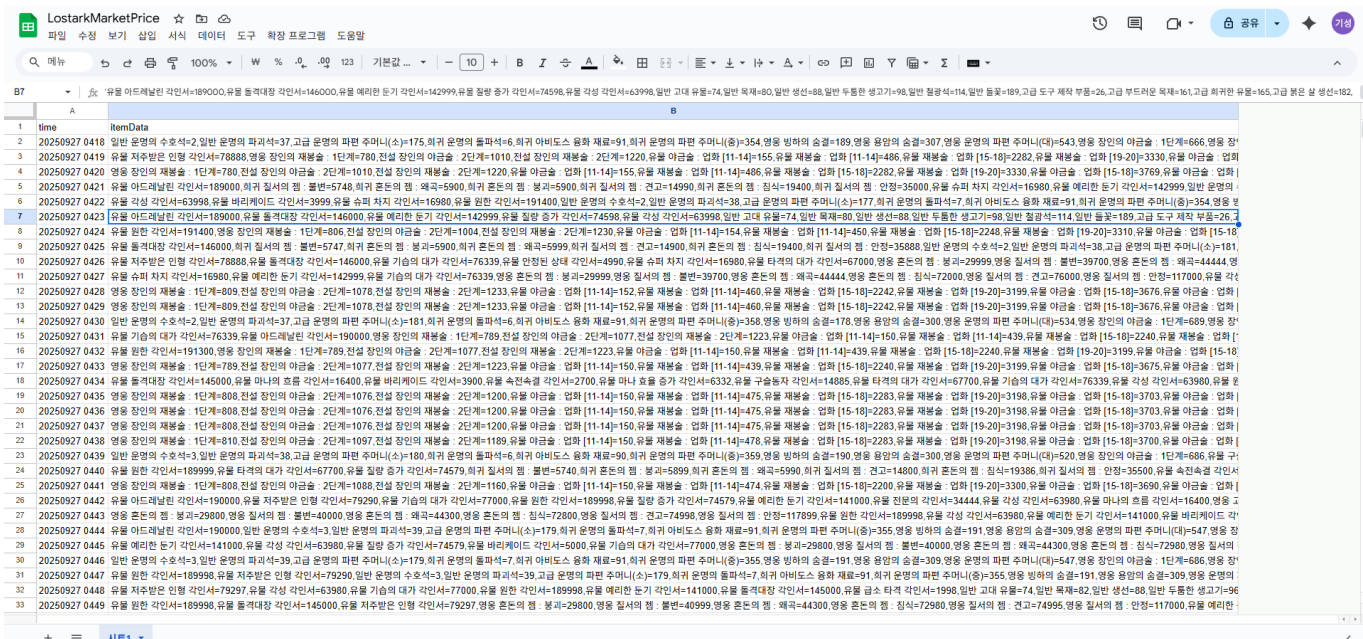
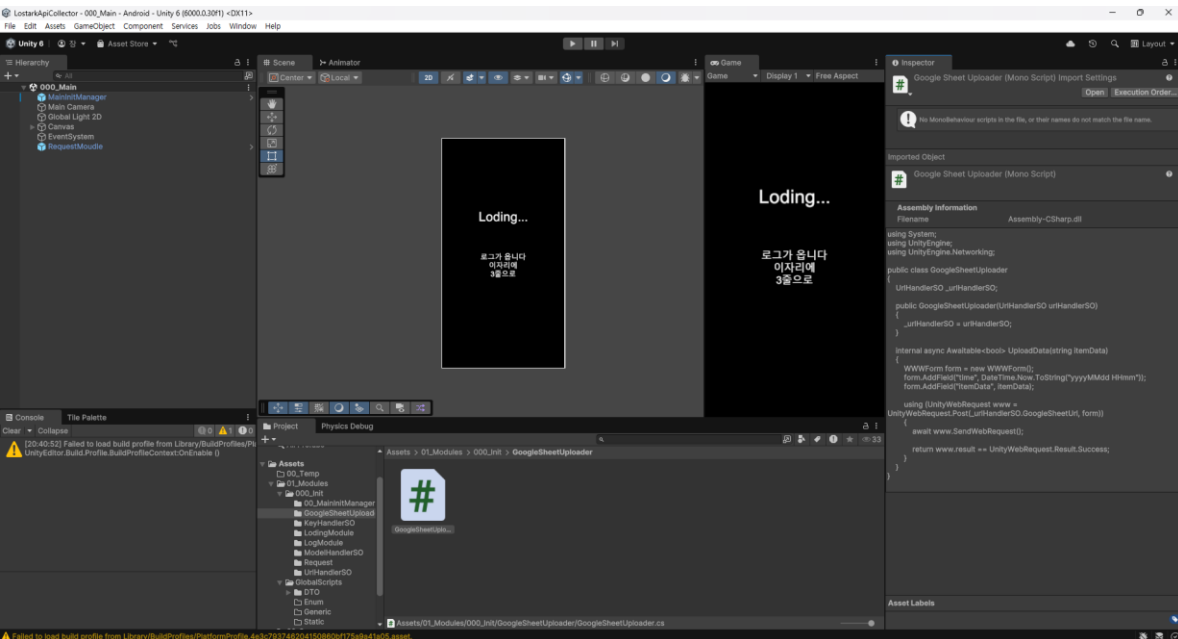
아이템 이름	현재 가격	최저 가격	수량
유물 강화 각인서	237930.3	238,000	1
유물 이드바실론 각인서	237671.3	235,999	3
유물 불타는 각인서	176902.5	180,499	1
유물 예리한 둔기 각인서	159137.8	159,000	1
유물 강화 증가 각인서	90632.3	92,222	2
유물 치수받은 안경 각인서	86111.3	87,500	1
유물 기술의 대가 각인서	84938.6	85,300	2
유물 각종 각인서	71599.6	72,700	1
유물 타격의 대가 각인서	60077.0	60,233	1
유물 전술의 각인서	38989.0	44,995	1

아이템 이름	현재 가격	최저 가격	수량
혼돈의 열: 침식	3970.5	3,879	2
침식의 열: 안정	9049.5	7,940	1
침식의 열: 불변	22634.5	20,000	1
혼돈의 열: 불기	16669.2	21,600	1
침식의 열: 안정	56202.2	51,500	3
혼돈의 열: 침식	50037.3	51,999	1
혼돈의 열: 파괴	44498.5	54,999	1
침식의 열: 견고	50643.4	55,886	1

아이템 이름	현재 가격	최저 가격	수량
10레벨 작업의 보석	1640	-	23시간 59분
10레벨 작업의 보석	1640	-	23시간 16분
10레벨 작업의 보석	1640	-	23시간 16분
10레벨 작업의 보석	1640	-	23시간 16분
10레벨 작업의 보석	1640	-	23시간 16분
10레벨 작업의 보석	1640	-	23시간 16분
10레벨 작업의 보석	1640	-	22시간 32분
10레벨 작업의 보석	1640	-	22시간 32분
10레벨 작업의 보석	1640	-	21시간 52분
10레벨 작업의 보석	1640	-	21시간 52분
10레벨 작업의 보석	1640	-	21시간 52분

데이터 수집 시스템 구축과정

- 유니티와 구글 스프레드시트를 사용하여 24시간 데이터를 수집하는 시스템 구축 계획



데이터 수집 시스템 구축과정

- 시퀀스 구조
- 1분마다 데이터 요청
- 데이터 받음
- 데이터 가공
- 구글 스프레드시트에 데이터 전송
- 데이터 저장

데이터 수집 시스템 구축과정

- SetAlarm() 호출시 현재시간이 20시 24분 12초라고 가정
- 초 단위는 무시하고 20시 25분을 다음 알람시간으로 설정

```
public class AlarmModule
{
    DateTime _now;
    참조 2개
    internal DateTime Alarm { get; private set; }

    참조 1개
    public AlarmModule()
    {
        SetAlarm();
    }

    참조 2개
    internal void SetAlarm()
    {
        _now = DateTime.Now;
        Alarm = new DateTime(_now.Ticks - (_now.Ticks % TimeSpan.TicksPerMinute) + TimeSpan.TicksPerMinute);
    }
}
```

데이터 수집 시스템 구축과정

- 1초에 50번 실행되는 함수
- 현재시간이 알람시간을 넘겼으면 다음 알람을 설정하고 API 요청을 날림

```
private void FixedUpdate()
{
    if (!_routineStartable) { return; }

    if (_alarmModule.Alarm < DateTime.Now)
    {
        _alarmModule.SetAlarm();

        _requestCall = 0;
        _dataBundle.Clear();

        for (int i = 0; i < _formDataArr.Length; i++)
        {
            ApiDataProcess(_formDataArr[i]).Cancel();
        }
    }
}
```

데이터 수집 시스템 구축과정

- Post요청을 보내고 데이터를 리스트에 추가
- 데이터가 전부 들어왔으면 구글 스프레드시트에 데이터를 전송하는데 나중에 기술

```
async Awaitable ApiDataProcess(FormData formData)
{
    _dataBundle.AddRange(await _requestMoudle.Post(formData));
    _requestCall++;

    if (_requestCall >= _maxCall)
    {
        string logFormat = DateTime.Now.ToString("yyyy-MM-dd\nHH:mm:ss")
            + (await _googleSheetUploader.UploadData(string.Join(',', _dataBundle))
            ? "\n전송완료"
            : "\n처리실패");

        _modelHandlerSO.LogModel.LogText = logFormat;
    }
}
```

데이터 수집 시스템 구축과정

- Post로 요청할 Form을 작성

```
internal FormData[] GetFormArr()
{
    FormData[] formArr = new FormData[] {
        // 강화재료
        new FormData(){
            urlType = UrlType.Market,
            formDto = new FormDto(){
                CategoryCode="50000",
                ItemTier="4",
                PageNo="1"
            }
        },
        new FormData()...,
        // 젼
        new FormData(){
            urlType = UrlType.Market,
            formDto = new FormDto()
            {
                CategoryCode="230000",
                ItemGrade="희귀",
                PageNo="1"
            }
        },
    },
}
```

```
각인서 상위 20개
// 생활
new FormData(){
    urlType = UrlType.Market,
    formDto = new FormDto()
    {
        CategoryCode="90000",
        PageNo="1"
    }
},
new FormData()...,
new FormData()...,
new FormData()...,
// 보석
new FormData(){
    urlType = UrlType.Auction,
    formDto = new FormDto()
    {
        Sort="BUY_PRICE",
        CategoryCode="210000",
        ItemTier="4",
        ItemName="5레벨",
        PageNo="1",
        SortCondition="ASC"
    }
},
},
```


데이터 수집 시스템 구축과정

- 로스트아크 API 특징
- URL에 따라 얻을 수 있는 데이터의 그룹이 나뉘어져 있음
 - 공지, 캐릭터 간단정보, 캐릭터 상세정보, 거래소, 경매장, 이벤트
- 그 중 두 개의 그룹을 사용하였음
 - 거래소 : 강화재료, 강화보조재료, 젼, 각인서, 생활재료
 - 경매장 : 보석

데이터 수집 시스템 구축과정

- 거래소 API 특징

- 요청을 보낼때의 항목

- 정렬기준, 카테고리 코드, 아이템등급, 아이템 이름,
- 아이템 희귀도, 페이지 번호, 정렬순서

- 받은 데이터의 항목

- 옆의 json 데이터 형태로 옴
- “Items” 리스트 내에 최대 10개의 아이템 정보가 옴
- 사용되는 항목
 - Name, Grade, CurrentMinPrice(현재최저가)

```
• {  
•   "PageNo": 1,  
•   "PageSize": 10,  
•   "TotalCount": 19,  
•   "Items": [  
•     {  
•       "Id": 66102106,  
•       "Name": "운명의 수호석",  
•       "Grade": "일반",  
•       "Icon": "https://cdn-lostark.game.onstove.com/efui_iconatlas/use/use_12_89.png",  
•       "BundleCount": 100,  
•       "TradeRemainCount": null,  
•       "YDayAvgPrice": 2.2,  
•       "RecentPrice": 3,  
•       "CurrentMinPrice": 2  
•     }, (이 외 10개항목이 추가로 더 옴)  
•   ]  
• }
```

데이터 수집 시스템 구축과정

- 경매장 API 특징
- 요청을 보낼때의 항목은 거래소와 동일
- 받은 데이터의 항목중 거래소와 차이점
 - 시작입찰가, 즉시구매가, 현재입찰가, 거래종료일 등
 - 사용되는 항목
 - Name, Grade, BuyPrice(즉시구매가)

```
{
  "PageNo": 1, "PageSize": 10, "TotalCount": 263,
  "Items": [
    {
      "Name": "8레벨 작열의 보석", "Grade": "유물", "Tier": 4, "Level": 1640,
      "Icon": "https://cdn-lostark.game.onstove.com/efui_iconatlas/use/use_12_113.png",
      "GradeQuality": null,
      "AuctionInfo": {
        "StartPrice": 294000, "BuyPrice": 294000, "BidPrice": 0,
        "EndDate": "2025-12-07T20:47:21.89", "BidCount": 0,
        "BidStartPrice": 294000, "IsCompetitive": false,
        "TradeAllowCount": 0, "UpgradeLevel": null
      },
      "Options": [
        {
          "Type": "GEM_SKILL_COOLDOWN_REDUCTION", "OptionName": "여우 불꽃",
          "OptionNameTripod": "", "Value": 20.0, "IsPenalty": false,
          "ClassName": "", "IsValuePercentage": true
        }
      ]
    }, (이 외 10개항목이 추가로 더 있음)
  ]
}
```

데이터 수집 시스템 구축과정

- 카테고리코드는 별도의 요청을 통해 기록해둠

```
"Code": 50000,  
"CodeName": "강화 재료"
```

```
"Code": 90000,  
"CodeName": "생활"
```

```
"Code": 40000,  
"CodeName": "각인서"
```

```
"Code": 230000,  
"CodeName": "아크 그리드 재료"
```

데이터 수집 시스템 구축과정

- 데이터 수집 - 강화재료



- 수집전략

- TotalCount = 19이고 요청시 한 페이지에 10개씩
- 2페이지만큼 요청

```
// 강화재료
new FormData(){
    urlType = UrlType.Market,
    formDto = new FormDto(){
        CategoryCode="50000",
        ItemTier="4",
        PageNo="1"
    }
},
new FormData(){
    urlType = UrlType.Market,
    formDto = new FormDto()
    {
        CategoryCode="50000",
        ItemTier="4",
        PageNo="2"
    }
},
```

데이터 수집 시스템 구축과정

- 데이터 수집 - 젬



- 수집전략

- 희귀도는 2개
 - 희귀, 영웅
- 희귀도당 TotalCount = 6 이고 요청시 한 페이지에 10개씩
 - 최대10개아이템 이기에 희귀도당 6개 아이템을 전부 받음
 - 희귀도 별 1회요청

```
// 젬
new FormData(){
    ...
    urlType = UrlType.Market,
    formDto = new FormDto()
    {
        CategoryCode="230000",
        ItemGrade="희귀",
        pageNo="1"
    }
},
new FormData(){
    ...
    urlType = UrlType.Market,
    formDto = new FormDto()
    {
        CategoryCode="230000",
        ItemGrade="영웅",
        pageNo="1"
    }
},
},
```

데이터 수집 시스템 구축과정

- 데이터 수집 - 각인서



- 수집전략

- 희귀도 - 유물 등급만이 유효
- 많은 종류의 각인서 중 실전에 쓰이는 상위 20개의 각인서만을 선별
- 각각 따로요청
 - 20회 요청

[illegible]

데이터 수집 시스템 구축과정

- 데이터 수집 - 생활재료



- 수집전략

- TotalCount = 35이고 요청시 한 페이지에 10개씩
- 4페이지만큼 요청

```
// 생활
new FormData(){
...
    urlType = UrlType.Market,
    formDto = new FormDto()
    {
        CategoryCode="90000",
        PageNo="1"
    }
},
new FormData()...,
new FormData()...,
new FormData(){
...
    urlType = UrlType.Market,
    formDto = new FormDto()
    {
        CategoryCode="90000",
        PageNo="4"
    }
},
```


데이터 수집 시스템 구축과정

- 데이터 수집 - 보석



- 수집전략

- 총 6개의 레벨에 대한 데이터 수집
 - 5, 6, 7, 8, 9, 10
- 4티어 보석만이 유효
- 즉시구매가 기준 오름차순으로 정렬
- 레벨별로 제일 저렴한 10개의 보석의 즉시구매가를 수집
 - 보석의 경우 빨리팔고 싶은 심리때문에
 - 터무니없이 낮은 가격에 올라오는 경우가 있음
 - 추후 하위 10개 보석의 평균가를 사용

```
// 보석
new FormData(){
    urlType = UrlType.Auction,
    formDto = new FormDto()
    {
        Sort="BUY_PRICE",
        CategoryCode="210000",
        ItemTier="4",
        ItemName="5레벨",
        PageNo="1",
        SortCondition="ASC"
    }
},
new FormData()...,
new FormData()...,
new FormData()...,
new FormData()...,
new FormData(){
    urlType = UrlType.Auction,
    formDto = new FormDto()
    {
        Sort="BUY_PRICE",
        CategoryCode="210000",
        ItemTier="4",
        ItemName="10레벨",
        PageNo="1",
        SortCondition="ASC"
    }
}
```

데이터 수집 시스템 구축과정

- 받은 json 데이터를 클래스로 치환
- 원하는 데이터만을 뽑아쓸 수 있게끔

```
public class MarketPage : IRequestDataToList
{
    public Items[] Items;

    참조 2개
    public List<string> GetList()
    {
        return Items.Select(item => item.ToString()).ToList();
    }
}
```

```
internal async Awaitable<List<string>> Post(FormData formData)
{
    string url = _urlHandlerSO.LostarkApiUrl[formData.urlType];
    WWWForm form = BuildForm(formData);

    using (UnityWebRequest www = UnityWebRequest.Post(url, form))
    {
        SetHeader(www);
        await www.SendWebRequest();

        return formData.urlType switch
        {
            UrlType.Market => JsonUtility.FromJson<MarketDto.MarketPage>(www.downloadHandler.text).GetList(),
            UrlType.Auction => JsonUtility.FromJson<AuctionDto.AuctionPage>(www.downloadHandler.text).GetList(),
            _ => new()
        };
    }
}
```

```
[Serializable]
참조 1개
public class Items
{
    public string Name;
    public string Grade;
    public string CurrentMinPrice;

    참조 1개
    public override string ToString()
    {
        string header = $"{Grade} ";
        if (Name.Contains("각인서")) { header = ""; }
        return header + $"{Name}={CurrentMinPrice}";
    }
}
```

데이터 수집 시스템 구축과정

- 아이템을 특정지을 수 있게끔 포맷
 - 등급 + 이름
- 그리고 뒤에 해당 아이템의 가격을 붙임

```
public class Items
{
    public string Name;
    public string Grade;
    public AuctionInfo AuctionInfo;

    참조 1개
    public override string ToString()
    {
        string header = $"{Grade} ";
        string[] bosuk = Name.Split();
        return header + $"{bosuk[0]} {bosuk[2]}={AuctionInfo.BuyPrice}";
    }
}
```

```
[Serializable]
참조 1개
public class Items
{
    public string Name;
    public string Grade;
    public string CurrentMinPrice;

    참조 1개
    public override string ToString()
    {
        string header = $"{Grade} ";
        if (Name.Contains("각인서")) { header = ""; }
        return header + $"{Name}={CurrentMinPrice}";
    }
}
```

데이터 수집 시스템 구축과정

- 구글 스프레드시트에 데이터를 전송

```
async Awaitable ApiDataProcess(FormData formData)
{
    _dataBundle.AddRange(await _requestMoudle.Post(formData));
    _requestCall++;

    if (_requestCall >= _maxCall)
    {
        string logFormat = DateTime.Now.ToString("yyyy-MM-dd\nHH:mm:ss")
            + (await _googleSheetUploader.UploadData(string.Join(',', _dataBundle))
            ? "\n전송완료"
            : "\n처리실패");

        _modelHandlerSO.LogModel.LogText = logFormat;
    }
}
```

데이터 수집 시스템 구축과정

- 사전에 약속된 이름의 어트리뷰트에 데이터 넣음
 - time = 현재시각("년년년년월월일일 시시분분" 형태)
 - ItemData = 아이템 정보("아이템1이름=가격, 아이템2이름=가격, ..., 아이템92이름=가격" 형태)
- 구글 스프레드시트에 데이터를 전송

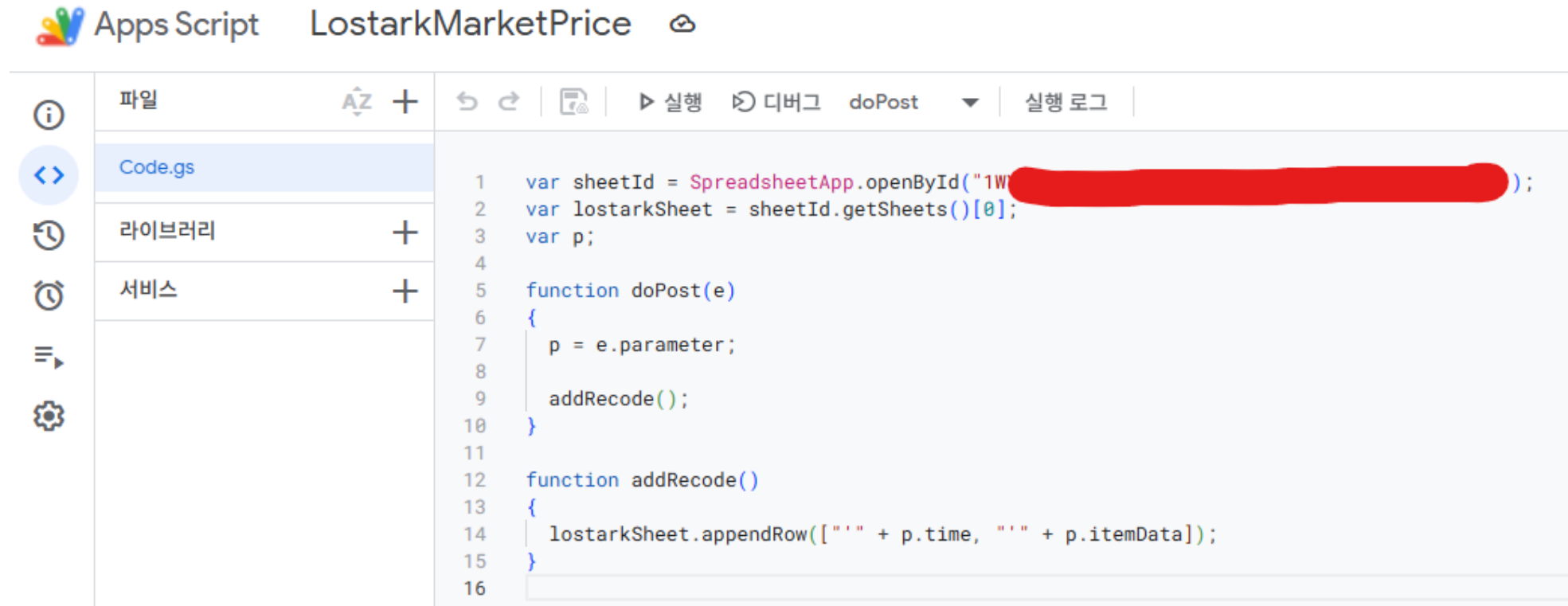
```
internal async Awaitable<bool> UploadData(string itemData)
{
    WWWForm form = new WWWForm();
    form.AddField("time", DateTime.Now.ToString("yyyyMMdd HHmm"));
    form.AddField("itemData", itemData);

    using (UnityWebRequest www = UnityWebRequest.Post(_urlHandlerSO.GoogleSheetUrl, form))
    {
        await www.SendWebRequest();

        return www.result == UnityWebRequest.Result.Success;
    }
}
```

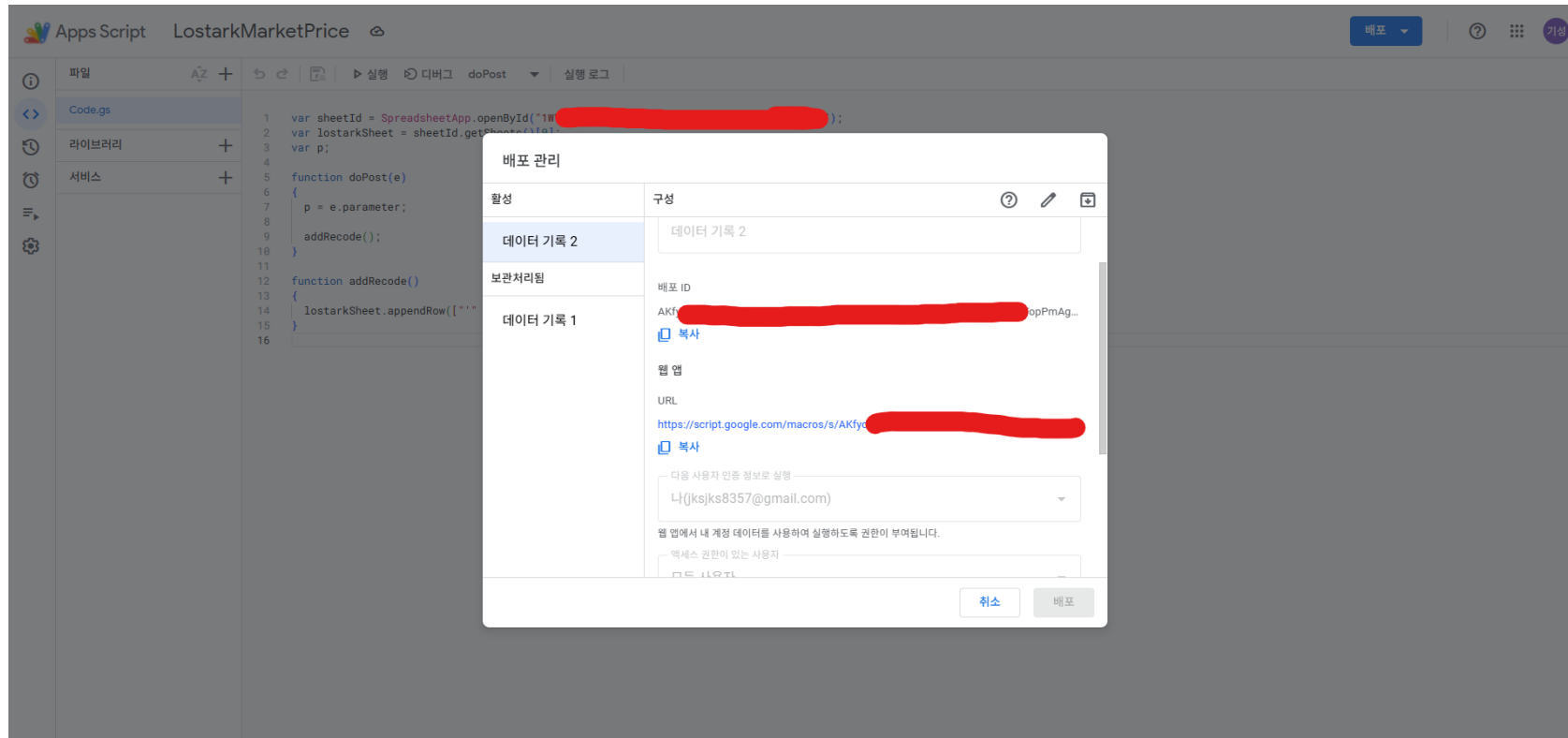
데이터 수집 시스템 구축과정

- Apps Script 를 통해 데이터 처리
 - time 및 itemData 어트리뷰트에 담긴 값을 스프레드시트에 한줄 추가한다
- AWS Lambda 와 유사한 기능



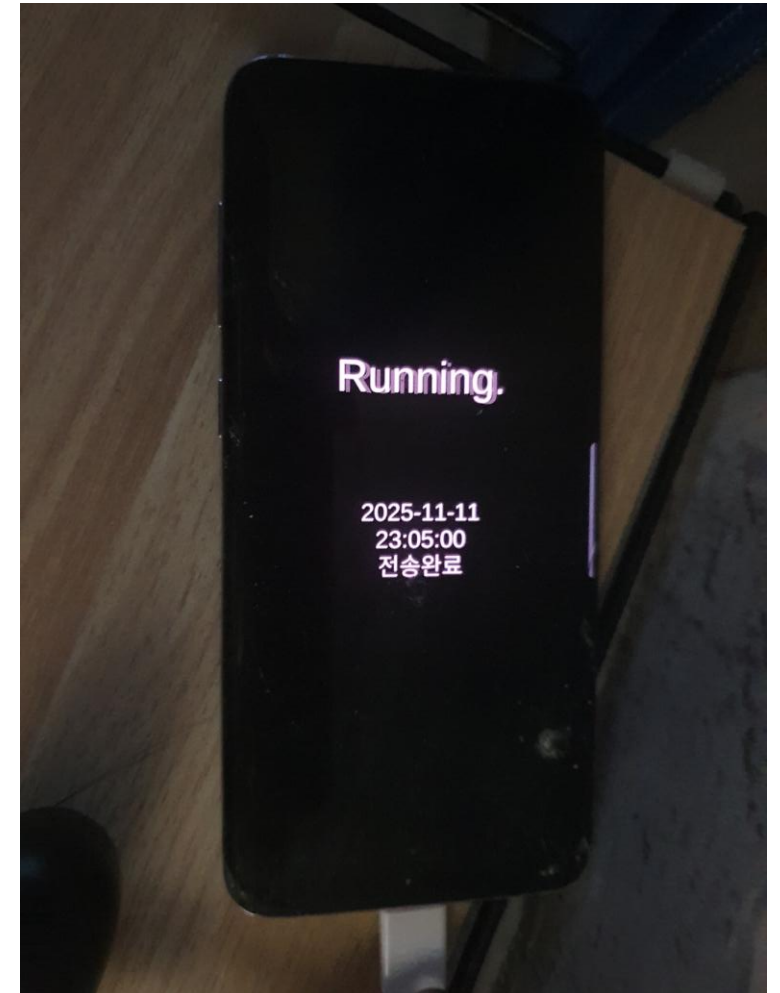
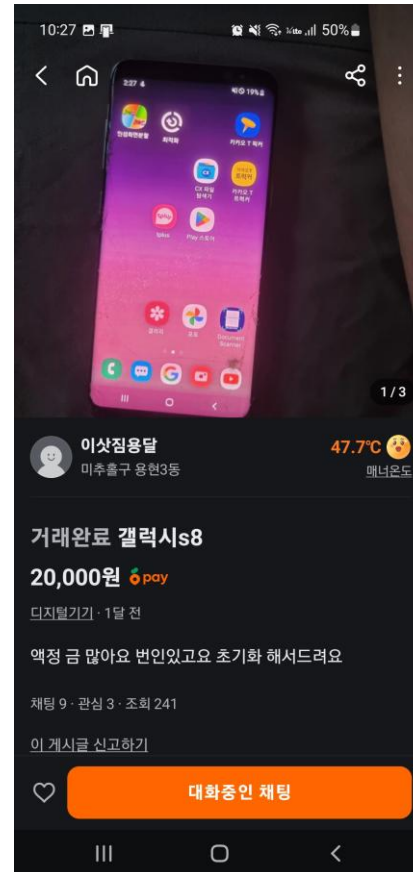
데이터 수집 시스템 구축과정

- Apps Script 를 통해 데이터 수신
- AWS API Gateway 와 유사한 기능



데이터 수집 시스템 구축과정

- 유니티 프로젝트를 .apk 파일로 빌드
- 공기계 구매 후 24시간 가동



전처리

- 판다스의 데이터프레임에 축적된 데이터를 넣으려 했으나
- 파이썬을 통해 포맷하여 넣는데 시간이 너무 오래걸림
- -> 데이터프레임에 넣기 좋은 형태로 양식을 변환하는 프로그램 개발

전처리

- 구글 스프레드시트는 API 요청을 통해
- csv, tsv 로 받을 수 있음
- 축적된 데이터를 다음과 같이
- lostarkData.json 파일로 변환
 - 결측부분은 “nan” 으로 채워짐
 - pd.read_json() 을 통해 바로 변환가능



Program.cs

- [
- {
- "date":"20250927 0418",
- "일반 운명의 수호석":"2",
- "일반 운명의 파괴석":"37",
- "고급 운명의 파편 주머니(소)":"175",
- ... ,
- "유물 8레벨 보석":"242228.9",
- "유물 9레벨 보석":"706950",
- "고대 10레벨 보석":"1949299.9"
- }, ...
-]

```
1 using Newtonsoft.Json;
2 using System;
3 using System.Collections.Generic;
4 using System.IO;
5 using System.Linq;
6 using System.Net.Http;
7 using System.Text;
8 using System.Threading.Tasks;
9
10 namespace SimpleTest
11 {
12     internal class Program
13     {
14         static async Task Main(string[] args)
15         {
16             string data;
17             using (var client = new HttpClient())
18             {
19                 var url = "https://docs.google.com/spreadsheets/d/1WYDQwP1H01WpR0D01hurfjy0ghvq770YgM2G8-1/export?format=csv";
20                 Console.WriteLine("다운 시작");
21                 var response = await client.GetAsync(url);
22                 Console.WriteLine("다운 시작");
23                 data = await response.Content.ReadAsStringAsync();
24                 Console.WriteLine("다운 완료");
25             }
26
27             List<Dictionary<string, string>> jsonDict = new List<Dictionary<string, string>>();
28             string[] recodes = data.Split('\n');
29
30             int j;
31             string[] curData, curItems, name_price;
32             Dictionary<string, List<int>> curItems = new Dictionary<string, List<int>>();
33             {
34                 ["전설 5레벨 보석"] = new List<int>(),
35                 ["전설 6레벨 보석"] = new List<int>(),
36                 ["전설 7레벨 보석"] = new List<int>(),
37                 ["유물 8레벨 보석"] = new List<int>(),
38                 ["유물 9레벨 보석"] = new List<int>(),
39                 ["고대 10레벨 보석"] = new List<int>()
40             };
41
42             int maxLength = recodes.Length;
43             for (int i = 1; i < maxLength; i++)
44             {
45                 Console.WriteLine($"{i}/{maxLength}");
46                 Dictionary<string, string> curRecode = new Dictionary<string, string>();
47
48                 curData = recodes[i].Split('\t');
49                 for (j = 0; j < curData.Length; j++)
50                 {
51                     curData[j] = curData[j].Trim();
52                 }
53
54                 curRecode["date"] = curData[0];
55
56                 curItems = curData[1].Split(',');
57                 ResetItems(curItems);
58
59                 for (j = 0; j < curItems.Length; j++)
60                 {
61                     name_price = curItems[j].Split('+');
62                     if (curItems.Contains(name_price[0]))
63                     {
64                         if (string.IsNullOrWhiteSpace(name_price[1]))
65                         {
66                             curItems[name_price[0]].Add(int.Parse(name_price[1]));
67                         }
68                     }
69                     else
70                     {
71                         curRecode[name_price[0]] = string.IsNullOrWhiteSpace(name_price[1]) ? "nan" : name_price[1];
72                     }
73                 }
74
75                 foreach (var item in curItems)
76                 {
77                     curRecode[item.Key] = item.Value.Count == 0 ? "nan" : (item.Value.Sum() / (double)item.Value.Count).ToString();
78                 }
79
80                 jsonDict.Add(curRecode);
81             }
82
83             using (StreamWriter sw = new StreamWriter("lostarkData.json"))
84             {
85                 sw.WriteLine(JsonConvert.SerializeObject(jsonDict));
86             }
87
88             static void ResetItems(Dictionary<string, List<int>> curItems)
89             {
90                 foreach (var item in curItems)
91                 {
92                     item.Value.Clear();
93                 }
94             }
95         }
96     }
97 }
```

전처리

- 데이터를 수집하는도중엔 결측치가 생김
 - 정기점검, 핸드폰 전원꺼짐 등

- 전처리

- 축적된 데이터의 맨 앞시간과 맨 뒷시간을 기준으로 새로운 데이터프레임 생성
 - 칼럼은 “date”, 위 조건 기준 1분간격으로 레코드 생성
- 새로운 데이터프레임에 축적된 데이터로 만든 데이터프레임을 left join
- “nan” -> numpy.nan 으로 치환
- 결측치는 ffill 처리
- 데이터 타입은 조금 가볍게 float64 -> float32 변경

```
temp_df = pd.read_json(loa_path)

one_min = pd.Timedelta(minutes=1)
head_time = temp_df.date[0]
tail_time = temp_df.date[len(temp_df.date) - 1]

df = []
while head_time <= tail_time:
    df.append(head_time)
    head_time += one_min
df = pd.DataFrame(df, columns=["date"])

df = df.merge(temp_df, left_on='date', right_on='date', how='left')
df = df.replace("nan", np.nan)
df = df.ffill()

for col in df.columns[1:]:
    df[col] = df[col].astype("float32")

print(df)
print("\n" * 5)
print(df.info())
```

전처리

- 데이터 준비 완료

<class 'pandas.core.frame.DataFrame'> - RangeIndex: 100743 entries, 0 to 100742 - Data columns (total 93 columns): # Column Non-Null Count Dtype --- --- 0 date 100743 non-null datetime64[ns] 1 일반 운명의 수호석 100743 non-null float32 2 일반 운명의 파괴석 100743 non-null float32 3 고급 운명의 파편 주머니(소) 100743 non-null float32 4 희귀 운명의 돌파석 100743 non-null float32 ... 90 유물 8레벨 보석 100743 non-null float32 91 유물 9레벨 보석 100743 non-null float32 92 고대 10레벨 보석 100743 non-null float32 - dtypes: datetime64[ns](1), float32(92) - memory usage: 36.1 MB - None	- date 일반 운명의 수호석 일반 운명의 파괴석 고급 운명의 파편 주머니(소) 희귀 운명의 돌파석 ... 유물 8레벨 보석 유물 9레벨 보석 고대 10레벨 보석 0 2025-09-27 04:18:00 2.0 37.0 175.0 6.0 ... 242228.906250 706950.0 1949299.875 1 2025-09-27 04:19:00 2.0 37.0 175.0 6.0 ... 242117.796875 706950.0 1949299.875 2 2025-09-27 04:20:00 2.0 37.0 175.0 6.0 ... 241637.796875 706950.0 1949299.875 3 2025-09-27 04:21:00 2.0 38.0 177.0 7.0 ... 241057.796875 706950.0 1949299.875 4 2025-09-27 04:22:00 2.0 38.0 177.0 7.0 ... 242129.000000 706950.0 1949299.875 ... 100738 2025-12-06 01:58:00 2.0 85.0 311.0 7.0 ... 292900.000000 857666.0 2385400.000 100739 2025-12-06 01:59:00 3.0 86.0 341.0 7.0 ... 292800.000000 857666.0 2385400.000 100740 2025-12-06 02:00:00 3.0 86.0 341.0 7.0 ... 292800.000000 857666.0 2385400.000 100741 2025-12-06 02:01:00 3.0 86.0 341.0 7.0 ... 292800.000000 857666.0 2385400.000 100742 2025-12-06 02:02:00 3.0 80.0 341.0 8.0 ... 292900.000000 857666.0 2385400.000 [100743 rows x 9 columns]
--	--

학습

- 모델-LSTM
- 시계열 데이터의 학습
 - 과거의 데이터 묶음을 인풋으로
 - 데이터 묶음의 마지막 시점의 바로 다음 시점의 데이터를 아웃풋으로 해야됨

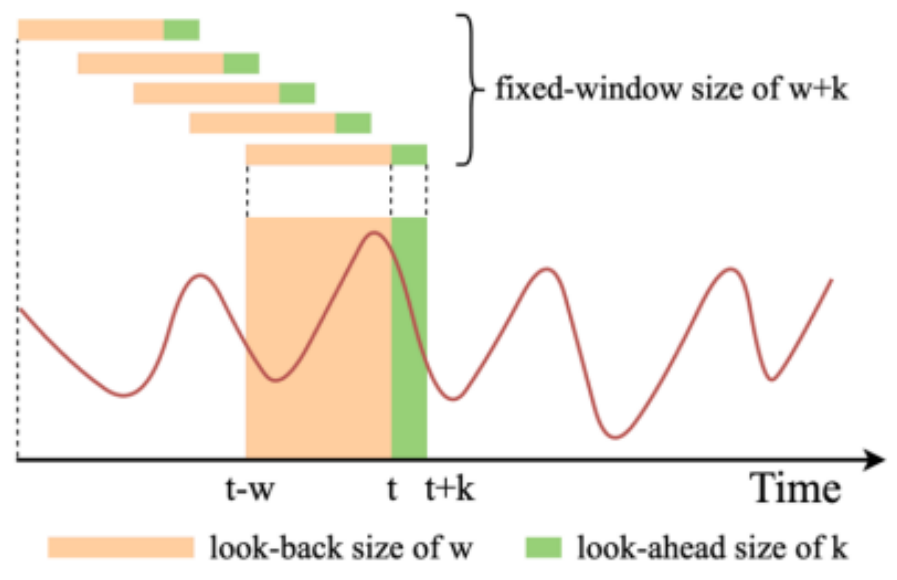


Fig. 1 Using a sliding window of width w to construct the training set for predicting k time steps in the future.

학습

- 모델 학습 전략 수립
- 그리드서치를 이용, 아래는 변화를 주는 파라미터
 - 데이터묶음 크기 (1~14분전, 1~30분전)
 - 퍼셉트론 개수 (64개, 128개)
 - 드롭아웃 비율 (10%, 20%)
- 2^3 경우의 수에 해당하는 모델의 성능평가

학습

- 데이터 묶음 30개, 퍼셉트론 128개, 드롭아웃 비율 10%
- 일 때 테스트 데이터를 통한 오차가 제일 낮음

	combo_name	window_size	lstm_units	dropout	batch_size	\
0	ws14_u128_d0.1	14	128	0.1	64	
1	ws30_u128_d0.1	30	128	0.1	64	
2	ws30_u64_d0.1	30	64	0.1	64	
3	ws14_u128_d0.2	14	128	0.2	64	
4	ws14_u64_d0.1	14	64	0.1	64	

	learning_rate	val_loss_min	test_rmse	test_mae	train_samples	\
0	0.001	0.000630	1122.791870	895.529358	70511	
1	0.001	0.000637	<u>915.095886</u>	<u>714.554871</u>	70501	
2	0.001	0.000843	1250.547974	994.820251	70501	
3	0.001	0.000852	1274.019043	1016.094910	70511	
4	0.001	0.000936	1427.211670	1156.726440	70511	

	val_samples	test_samples	elapsed_sec
0	15109	15109	381.900556
1	15106	15106	670.068000
2	15106	15106	377.059180
3	15109	15109	343.929144
4	15109	15109	210.835756

학습

- 모델 학습 전략 수립 2
- 그리드서치를 이용, 아래는 변화를 주는 파라미터
 - 데이터묶음 크기 (1~14분전, 1~30분전, 1~45분전, 1~60분전)

학습

- 데이터 묶음 크기가 클 수록 학습률은 증가
- 하지만 예측데이터와 테스트간 오차는 데이터묶음 크기가 30일때 제일낮다
- 데이터 묶음 크기가 큰 경우엔 과적합이 의심됨

	combo_name	window_size	lstm_units	dropout	batch_size	\
0	ws14_u128_d0.1	14	128	0.1	64	
1	ws60_u128_d0.1	60	128	0.1	64	
2	ws45_u128_d0.1	45	128	0.1	64	
3	ws30_u128_d0.1	30	128	0.1	64	

	learning_rate	val_loss_min	test_rmse	test_mae	train_samples	\
0	0.001	0.000679	1101.987305	876.159363	70511	
1	0.001	0.000726	1086.053223	868.317444	70479	
2	0.001	0.000747	1082.980591	873.162964	70490	
3	0.001	0.000789	<u>1077.100342</u>	<u>862.179077</u>	70501	

	val_samples	test_samples	elapsed_sec
0	15109	15109	559.665469
1	15102	15102	1297.890395
2	15104	15104	1539.231730
3	15106	15106	981.120904

학습

- 최종 하이퍼 파라미터
- 데이터묶음 30개(1~30분 전의 1분간격 데이터를 기준으로 그 다음시간의 가격예측)
- 검증데이터 비율 15%
- 테스트데이터 비율 15%
- 드롭아웃 비율 10%
- 퍼셉트론 개수 128
- 배치사이즈 128
- 그 외 항목들
 - 밀가루 가격이 올라가면 빵, 면 등의 가격이 올라가듯 상관관계를 고려하여 92개 데이터 동시학습
 - Loss가 2회연속 감소하지 않으면 조기종료
 - 데이터를 0~1 크기로 정규화

결과

- 그래프의 모습을 보면 전체적으로 학습이 관찮게 됬음
- 드롭박스의 맨 위 항목에 전체보기가 있음
- 시각화 사이트
 - https://terrerrrrra.github.io/Inha2025_BigData/

결과

- 미래의 가격을 예측하기 위해선
- 현재시점의 가격데이터 포함 30분전의 가격데이터를 1분간격의 묶음으로
- 1분후의 예측데이터를 얻고
- 1분후의 데이터 + 29분전의 데이터를 통해 2분후 데이터를 얻는다
- 이를 반복하여 미래의 데이터를 예측 할 수 있음

기대효과

- 학습된 모델을 이용하여 아이템가격의 향후시세를 미리 볼 수 있다면
- 쌀때 사들이고 비쌀때 판매하여 이득을 취할 수 있다

A top-down view of various gaming peripherals arranged on a dark, textured wooden surface. In the upper center is a black game controller with a white directional pad and several buttons. To its right is a black wired mouse with blue LED lighting on its side. In the upper right corner, a portion of a black keyboard is visible, showing keys like '5', '6', and '7'. On the left side, another black keyboard is partially shown, featuring keys like 'PRINT SCREEN', 'SCR LK', and 'PAUSE BREAK'. Below the controller is a small black webcam on a stand. In the bottom left corner, a black mouse is visible. In the bottom right corner, a black headset with red and black braided headband is shown. The text '감사합니다' is centered in the middle of the image in a large, white, sans-serif font.

감사합니다